

Alberi binari

Luca Didaci

Table of contents

1	Alberi binari	1
1.1	Alberi binari	1
1.2	Attraversamento alberi	3
1.3	PREORDER: nodo, sottoalbero sx, sottoalbero dx	3
1.4	INORDER: sottoalbero sx, nodo, sottoalbero dx	3
1.5	POSTORDER: sottoalbero sx, sottoalbero dx, nodo	4
1.6	Trovare la profondità dell'albero	4
1.7	Implementazione non ricorsiva	8
1.8	PREORDER: nodo, sottoalbero sx, sottoalbero dx	8
1.9	INORDER: sottoalbero sx, nodo, sottoalbero dx	9
1.10	POSTORDER: sottoalbero sx, sottoalbero dx, nodo	10
1.11	Trovare la profondità dell'albero	11

1 Alberi binari

1.1 Alberi binari

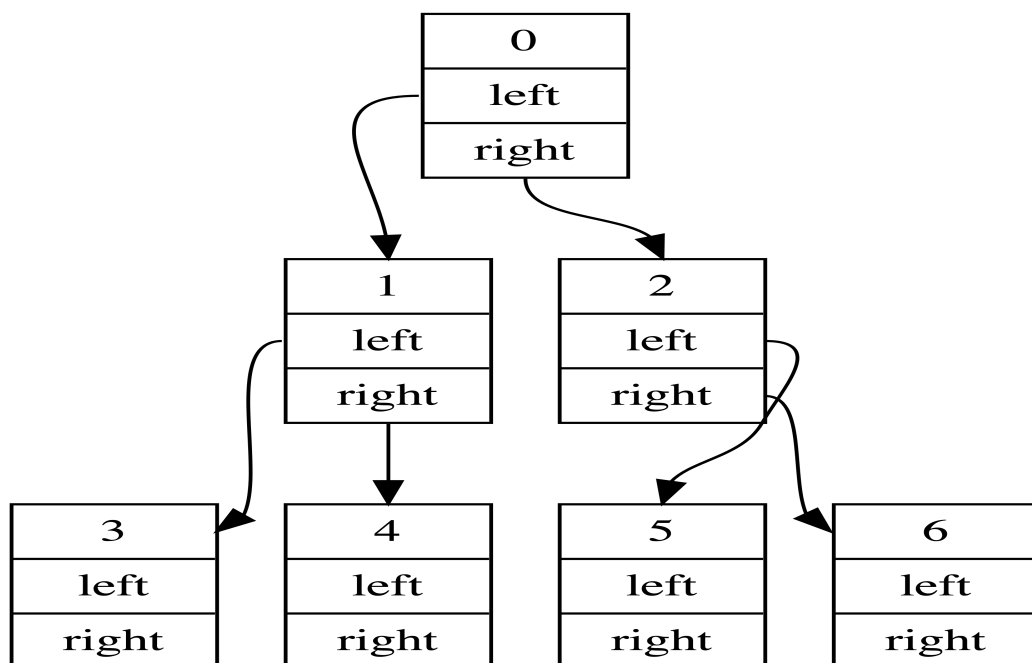
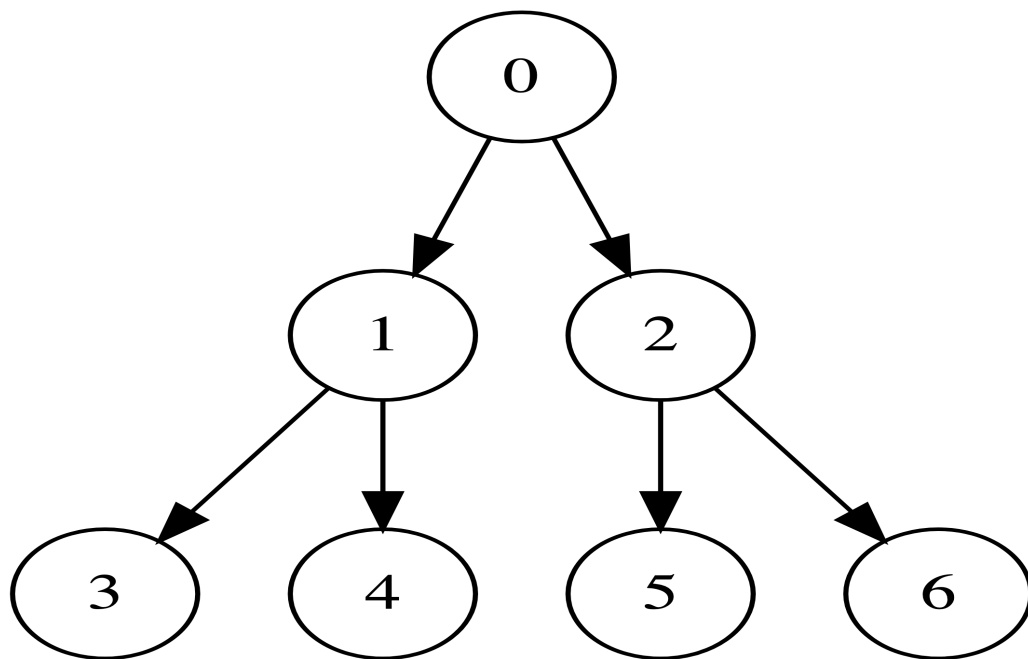
- Struttura dati

Possiamo implementare un albero binario mediante una struttura che rappresenta il singolo nodo. Il nodo contiene un campo 'valore' e due puntatori, rispettivamente al sottoalbero sinistro e al sottoalbero destro

```
typedef struct node *Node;
// Node è un puntatore alla struct node

struct node{ Item item; Node left, right};
```

- è possibile aggiungere un puntatore al nodo **parent** per rendere agevole percorrere l'albero dalle foglie alla radice
- alcune operazioni sono elementari: inserire/cancellare un nodo foglia; combinare due alberi come ramo destro e ramo sinistro di una nuova radice; spostarsi da un nodo a uno dei figli
- alcune operazioni sono meno elementari, perché occorre gestire i puntatori 'appesi', per esempio cancellare un nodo *non* foglia (occorre gestire 2 figli e 1 padre)



1.2 Attraversamento alberi

1.3 PREORDER: nodo, sottoalbero sx, sottoalbero dx

Esploro l'albero seguendo quest'ordine

- analizzo il nodo
- analizzo (ricorsivamente) il sottoalbero sinistro, passandolo alla funzione `traverse_preorder`
- analizzo (ricorsivamente) il sottoalbero destro, passandolo alla funzione `traverse_preorder`

notate il criterio di stop, o **caso base**, della struttura ricorsiva (altrimenti la ricorsione diventa infinita).

```
void traverse_preorder(Node h, void (*visit) (Node)){
    if (h== NULL) return;
    visit(h); //printf("[ %d ]",h->item);
    traverse_preorder(h->left,visit );
    traverse_preorder(h->right,visit );
}
```

1.4 INORDER: sottoalbero sx, nodo, sottoalbero dx

Il ragionamento è analogo:

- richiamo la funzione sul sottoalbero sinistro
- visito il nodo
- richiamo la funzione sul sottoalbero destro

```
void traverse_inorder(Node h, void (*visit) (Node)){
    if (h== NULL) return;
    traverse_inorder(h->left,visit );
    visit(h); //printf("[ %d ]",h->item);
    traverse_inorder(h->right,visit );
}
```

1.5 POSTORDER: sottoalbero sx, sottoalbero dx, nodo

Il ragionamento è analogo:

- richiamo la funzione sul sottoalbero sinistro
- richiamo la funzione sul sottoalbero destro
- visito il nodo

```
void traverse_postorder(Node h, void (*visit) (Node)){  
  
    if (h== NULL) return;  
    traverse_postorder(h->left,visit );  
    traverse_postorder(h->right,visit );  
    visit(h); //printf("[ %d ]",h->item);  
}
```

(provate con carta e penna a seguire il flusso delle ricorsioni)

Posso passare alla funzione **traverse** il puntatore alla funzione per la ‘visita’ del nodo. In questo modo posso adattare il codice facilmente, nel caso in cui il valore del nodo non sia un intero ma una struttura con vari campi (nome, cognome, ID, ...). Inoltre posso usare la stessa struttura della funzione **traverse** per modificare il valore del nodo

```
void show_node(Node h){ printf("[ %d ]",h->item); }  
void set_0(Node h){ h->item=0; }  
  
//....  
  
traverse_preorder(n0, show_node);  
  
//....  
  
traverse_preorder(n0, set_0);
```

1.6 Trovare la profondità dell'albero

Implementazione ricorsiva:

- Se il nodo è NULL l'altezza è 0
- altrimenti l'altezza è 1 + l'altezza dell'albero che si diparte da quel nodo


```
int tree_height_recursive(Node h) {
    if (h == NULL)
        return 0;
    return 1 + max(
        tree_height_recursive(h->left),
        tree_height_recursive(h->right)
    );
}
```

NOTA: questa funzione si può applicare anche ad alberi binari non completi e non bilanciati. Se l'albero invece viene riempito 'da sinistra' come negli HEAP (alberi binari completi o semicompleti) allora è sufficiente esplorare il ramo sinistro.

```
int tree_height_recursive(Node h) {
    if (h == NULL)
        return 0;
    return 1 + tree_height_recursive(h->left);
}
```

Il codice allegato:

- crea un albero binario con i nodi `n0`,..., `n6` inizializzando i nodi e collegandoli tra loro
- esplora l'albero usando le tre funzioni `traverse_...()` ricorsive
- utilizza la funzione `traverse_postorder()` con la funzione `set_0()` per impostare a 0 tutti i nodi
- utilizza la funzione `tree_height`, in forma, ricorsiva, per determinare la profondità dell'albero.

```
#include <stdio.h>
#include <stdlib.h>

typedef int Item;
// in modo da memorizzare interi o altro

typedef struct node *Node;
// Node è un puntatore alla struct node
```

```

struct node{ Item item; Node left, right;};

Node newNode(Item item) {
    Node p = malloc(sizeof *p);
    if (p == NULL) {
        perror("malloc");
        exit(EXIT_FAILURE);
    }
    p->item = item;
    p->left = NULL;
    p->right = NULL;
    return p;
}

void show_node(Node h){
    printf("[ %d ]",h->item);
}

void set_0(Node h){
    h->item=0;
}

void traverse_preorder(Node h, void (*visit) (Node)){

    if (h== NULL) return;
    visit(h); //printf("[ %d ]",h->item);
    traverse_preorder(h->left,visit );
    traverse_preorder(h->right,visit );
}

void traverse_inorder(Node h, void (*visit) (Node)){

    if (h== NULL) return;
    traverse_inorder(h->left,visit );
    visit(h); //printf("[ %d ]",h->item);
    traverse_inorder(h->right,visit );
}

void traverse_postorder(Node h, void (*visit) (Node)){

    if (h== NULL) return;
    traverse_postorder(h->left,visit );
    traverse_postorder(h->right,visit );
    visit(h); //printf("[ %d ]",h->item);
}

```

```

}

int max(int a, int b) {
    return (a > b) ? a : b;
}

int tree_height_recursive(Node h) {
    if (h == NULL)
        return 0;
    return 1 + max(
        tree_height_recursive(h->left),
        tree_height_recursive(h->right)
    );
}

int main(void) {
    Node n0, n1, n2, n3, n4, n5, n6;
    n0 = newNode(0);
    n1 = newNode(1);
    n2 = newNode(2);
    n3 = newNode(3);
    n4 = newNode(4);
    n5 = newNode(5);
    n6 = newNode(6);

    n0->left=n1;
    n0->right=n2;
    n1->left=n3;
    n1->right=n4;
    n2->left=n5;
    n2->right=n6;

    printf("Livello 0: %d\n", n0->item);
    printf("Livello 1: %d %d\n", n1->item, n2->item);
    printf("Livello 2: %d %d %d %d\n", n3->item, n4->item, n5->item, n6->item );
    printf("\n");

    printf("Livello 0: %d\n", n0->item);
    printf("Livello 1: %d %d\n", n0->left->item, n0->right->item);
    printf("Livello 2: %d %d %d %d\n",
        n0->left->left->item, n0->left->right->item,
        n0->right->left->item, n0->right->right->item );

    printf("\nPREORDER\n");
    traverse_preorder(n0, show_node);
}

```

```
printf("\nINORDER\n");
traverse_inorder(n0, show_node);

printf("\nPOSTORDER\n");
traverse_postorder(n0, show_node);

printf("\nset 0\n");
traverse_postorder(n0, set_0);
traverse_postorder(n0, show_node);

printf("\nProfondità: %d\n", tree_height_recursive(n0 ) );

return 0;
}
```

1.7 Implementazione non ricorsiva

1.8 PREORDER: nodo, sottoalbero sx, sottoalbero dx

Per implementare in modo non ricorsivo posso procedere come segue:

- visito il nodo
- metto in stack il sottoalbero DESTRO (perché lo voglio esplorare dopo il sinistro)
- metto in stack il sottoalbero SINISTRO (in modo che sia lui a trovarsi in cima allo stack e venga processato subito dopo.)
- estraggo dallo stack il prossimo nodo e applico su di esso gli stessi passi.

```
if (!root) return;

push( root);
while (!stack_is_empty(st)) {
    h = POP();
    visit(h);
    PUSH( h->right);
    PUSH( h->left);
}
```

Sequenza di visita ottenuta:

[0][1][3][4][2][5][6]

1.9 INORDER: sottoalbero sx, nodo, sottoalbero dx

Contenuto dello stack durante i vari passaggi (ogni colonna rappresenta uno stato dello stack)

						3							
			1		4	4	4				5		
0		2	2	2	2	2	2	2		6	6	6	

1.9 INORDER: sottoalbero sx, nodo, sottoalbero dx

Dato il puntatore `h`, la prima azione è esplorare il sottoalbero sinistro `h->left`, che significa (lo sappiamo dall'implementazione ricorsiva, che è la più naturale)

- esplorare `h->left->left`
- visitare `h->left`
- esplorare `h->left->right`

Ma esplorare INORDER `h->left->left` significa

- esplorare `h->left->left->left`
- visitare `h->left->left`
- esplorare `h->left->right->right ...` e così via.

Dunque devo 'scendere' fino a che è possibile sul ramo sinistro, poi visito il nodo, poi mi occupo del ramo destro. I nodi vengono messi in stack per visitarli successivamente.

Per ottenere la sequenza di azioni desiderata ma senza usare la ricorsione posso procedere in questo modo

Indichiamo con `curr` il nodo corrente

- Scendi lungo il ramo sinistro:
 - finché `curr != NULL`, metti `curr` sullo stack per esaminarlo in seguito e spostati a sinistra.
- Estrai dallo stack il nodo in cima, visitalo.
- Passa al suo ramo destro (assegnando `curr = nodo->right`) e ricomincia

1 Alberi binari

Sequenza di visita ottenuta:

[3] [1] [4] [0] [5] [2] [6]

Contenuto dello stack durante i vari passaggi (ogni colonna rappresenta uno stato dello stack)

		3								
	1	1	1		4			5		
0	0	0	0	0	0	0	2	2	2	

1.10 POSTORDER: sottoalbero sx, sottoalbero dx, nodo

Ricordiamo che in PREORDER l'ordine di visita è nodo, sottoalbero sx, sottoalbero dx.

Dunque se considero un PREORDER 'modificato' nodo, sottoalbero dx, sottoalbero sx ottengo la sequenza *opposta* a ciò che otterrei in post-order.

Sequendo questo ragionamento effettuo un PREORDER 'modificato' mettendo gli elementi in un secondo stack per invertirne l'ordine.

Creiamo dunque due stack, S1 e S2

1. push root $\rightarrow$ S1
2. while S1 non è vuoto:
 - pop h da S1 $\rightarrow$ push h su S2
 - se h->left, push su S1
 - se h->right, push su S1
3. while S2 non è vuoto:
 - pop da S2 e visita

Sequenza di visita ottenuta:

[3] [4] [1] [5] [6] [2] [0]

Contenuto degli stack durante i vari passaggi (ogni colonna rappresenta uno stato dello stack)

stack1							6							
				2		5	5	5				4		
	0		1	1	1	1	1	1	1		3	3	3	
stack2														3
													4	4
										1	1	1	1	1
									5	5	5	5	5	5
								6	6	6	6	6	6	6
					2	2	2	2	2	2	2	2	2	2
		0	0	0	0	0	0	0	0	0	0	0	0	0

1.11 Trovare la profondità dell'albero

Implementazione NON ricorsiva:

- Se il nodo è NULL l'altezza è 0
- altrimenti l'altezza è $1 + \text{l'altezza dell'albero che si diparte da quel nodo}$

```
int tree_height(Node h) {
    int altezza = 0;
    Node current = h;
    while (current != NULL) {
        altezza++;
        current = current->left;
    }
    return altezza;
}
```

NOTA: funziona SOLO SE l'albero viene riempito 'da sinistra' come negli HEAP (alberi binari completi o semicompleti). In caso contrario l'albero può essere sbilanciato ed il codice è più complesso (occorre usare una CODA per depositare i nodi da visitare successivamente)

Il codice allegato (draft, manca il codice dello stack) mostra l'implementazione delle funzioni illustrate senza l'uso della ricorsione

```

#include <stdio.h>
#include <stdlib.h>

typedef int Item;
// in modo da memorizzare interi o altro

typedef struct node *Node;
// Node è un puntatore alla struct node

struct node{ Item item; Node left, right;};

Node newNode(Item item) {
    Node p = malloc(sizeof *p);
    if (p == NULL) {
        perror("malloc");
        exit(EXIT_FAILURE);
    }
    p->item = item;
    p->left = NULL;
    p->right = NULL;
    return p;
}

void show_node(Node h){
    printf("[ %d ]",h->item);
}

void set_0(Node h){
    h->item=0;
}

void traverse_preorder(Node root, void (*visit)(Node)) {
    if (!root) return;
    Stack st = stack_create();
    stack_push(st, root);
    while (!stack_is_empty(st)) {
        Node h = stack_pop(st);
        visit(h);
        // Inserisci prima DX così SX viene processato prima
        if (h->right) stack_push(st, h->right);
        if (h->left) stack_push(st, h->left);
    }
    stack_destroy(st);
}

void traverse_inorder(Node root, void (*visit)(Node)) {

```



```

Stack st = stack_create();
Node curr = root;
while (curr || !stack_is_empty(st)) {
    // vai a sx alla massima profondità
    while (curr) {
        stack_push(st, curr);
        curr = curr->left;
    }
    // ora curr = NULL; torniamo indietro
    curr = stack_pop(st);
    visit(curr);
    // esplora sottoalbero DX
    curr = curr->right;
}
stack_destroy(st);
}

void traverse_postorder(Node root, void (*visit)(Node)) {
    if (!root) return;
    // Usa due stack
    Stack s1 = stack_create();
    Stack s2 = stack_create();
    stack_push(s1, root);
    while (!stack_is_empty(s1)) {
        Node h = stack_pop(s1);
        stack_push(s2, h);
        if (h->left) stack_push(s1, h->left);
        if (h->right) stack_push(s1, h->right);
    }
    // now s2 has nodes in reverse-postorder
    while (!stack_is_empty(s2)) {
        visit(stack_pop(s2));
    }
    stack_destroy(s1);
    stack_destroy(s2);
}

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main(void) {
    Node n0, n1, n2, n3, n4, n5, n6;
    n0 = newNode(0);
    n1 = newNode(1);
    n2 = newNode(2);
    n3 = newNode(3);

```

```

n4 = newNode(4);
n5 = newNode(5);
n6 = newNode(6);

n0->left=n1;
n0->right=n2;
n1->left=n3;
n1->right=n4;
n2->left=n5;
n2->right=n6;

printf("Livello 0: %d\n", n0->item);
printf("Livello 1: %d %d\n", n1->item, n2->item);
printf("Livello 2: %d %d %d %d\n", n3->item, n4->item, n5->item, n6->item );
printf("\n");

printf("Livello 0: %d\n", n0->item);
printf("Livello 1: %d %d\n", n0->left->item, n0->right->item);
printf("Livello 2: %d %d %d %d\n",
       n0->left->left->item, n0->left->right->item,
       n0->right->left->item, n0->right->right->item );

printf("\nPREORDER\n");
traverse_preorder(n0, show_node);

printf("\nINORDER\n");
traverse_inorder(n0, show_node);

printf("\nPOSTORDER\n");
traverse_postorder(n0, show_node);

printf("\nset 0\n");
traverse_postorder(n0, set_0);
traverse_postorder(n0, show_node);

printf("\nProfondità: %d\n", tree_height_recursive(n0 ) );

return 0;
}

```